

Audio Output

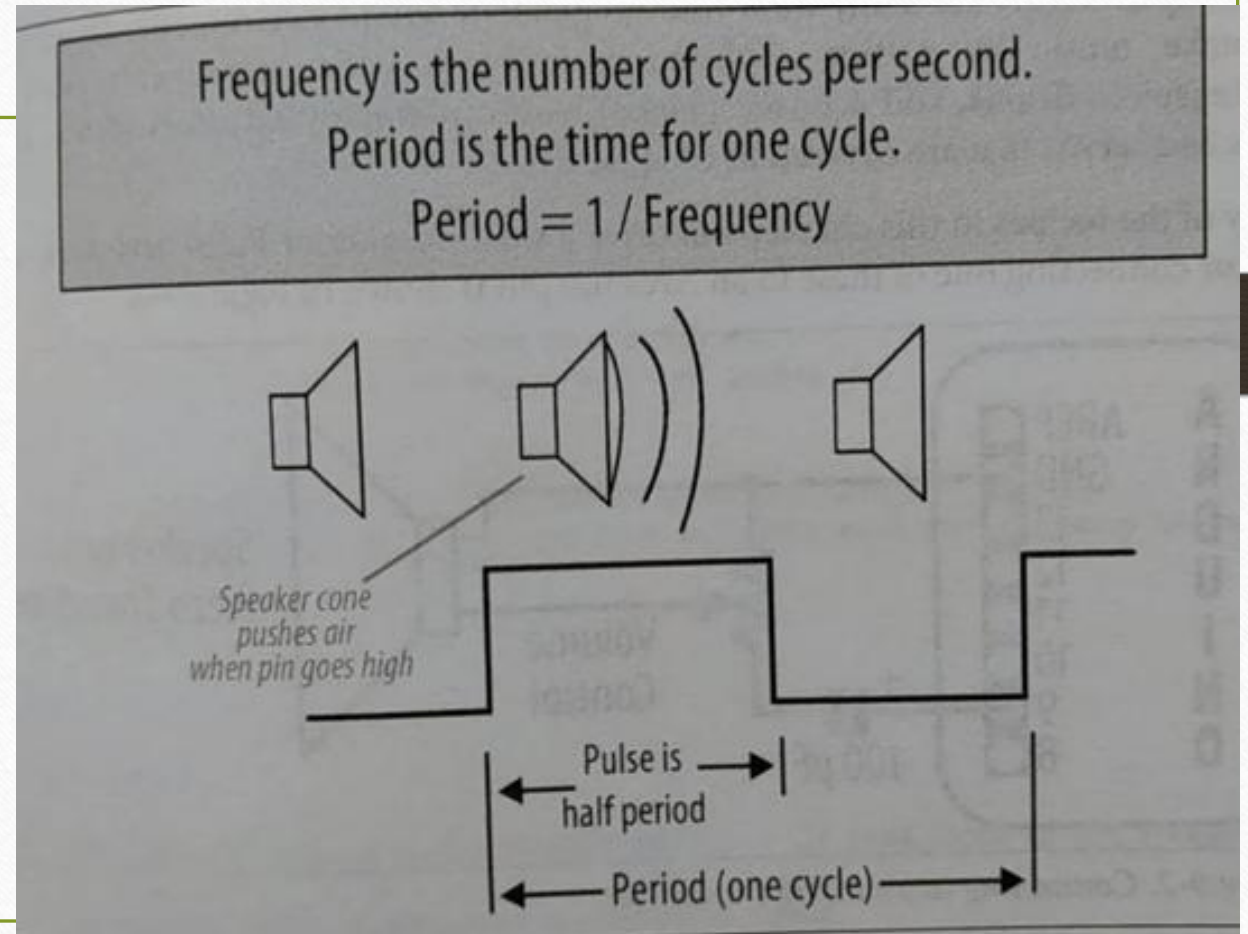
Chao-Chin Wu

Reference: M. Margolis, et al., Arduino Cookbook, 3rd Edition,
Oreilly, 2020

<https://github.com/bjepson/Arduino-Cookbook-3ed-INO/tree/master/ch09>

Introduction

- Sound is produced by **vibrating air**.
- A sound has a distinctive **pitch** if the vibration repeats regularly.
- The **pitch** (frequency) of the sound is determined by the time it takes to pulse the speaker in and out.



Introduction

- The shorter the amount of time, the higher the frequency.
- The unit of frequency is measured in **hertz**.

Introduction

- The Arduino can create sound by driving a **loudspeaker** or **Piezo** (壓電) device.
- Two types of Piezos
 - A Piezo speaker can produce sounds across a range frequencies.
 - A Piezo **buzzer** includes oscillator circuitry that cause it to buzz at a fixed frequency

Introduction

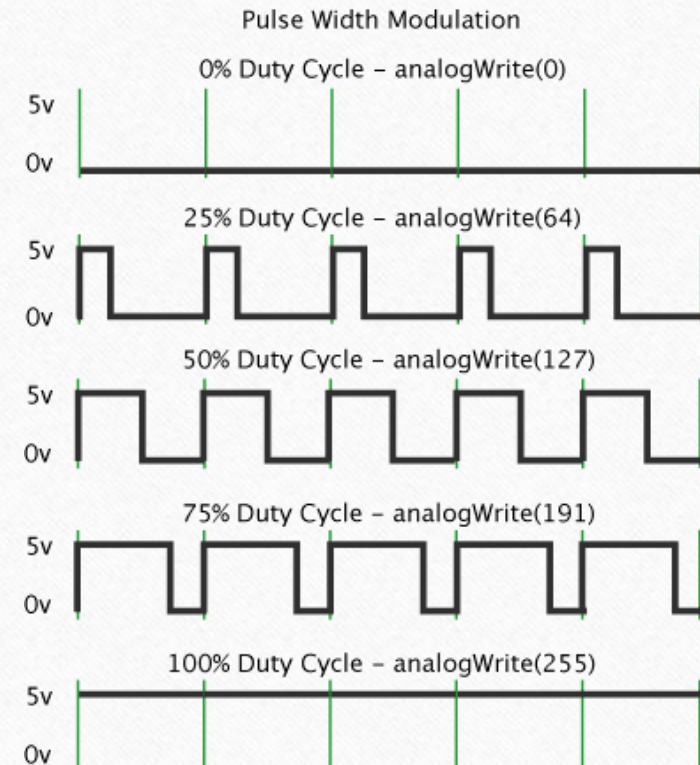
- The **tone** function is to produce sound using hardware **timer**.
- On a standard Arduino board, only one tone can be produced at a time.
- When you use the tone function, it will tie up the timer used for *analogWrite* on **pins 3 and 11**.
 - choose different pins if you need analog output.

Play Tone using Hardware Timer

-
- Arduino Uno has 3 timers (TIMER0, TIMER1, TIMER2), which is to control the PWM pins:
 - Pins 5 and 6: controlled by Timer 0
 - Pins 9 and 10: controlled by timer 1
 - Pins 11 and 3: controlled by timer 2

PWM Pins

- Pulse Width Modulation, or PWM, is a **technique for getting analog results with digital means.**
- Digital control is used to create a square wave, a signal switched between on and off.



-
- Pulse-width modulation (PWM) is a powerful technique for **controlling analog circuits with a microcontroller's digital outputs.**
 - PWM is used in many applications, ranging from **communications** to **power control** and **conversion.**

Can PWM pins used as input?

- Yes.
- The PWM pins (and in most cases the Analog IN pins) can be used as digital I/O pins (pinMode INPUT, INPUT_PULLUP, or OUTPUT).

Can analog pins be used as PWM?

- The pin number is its only parameter.
- The Arduino does not have a digital-to-analog converter (DAC) built-in, but **it can do pulse-width modulation (PWM)**
 - a digital signal used to achieve some of an analog output's functions.
- The function `analogWrite(pin, value)` is used to output a PWM signal.

Is PWM signal analog or digital?

- Pulse Width Modulation (PWM) is a method for encoding an **analog signal into a single digital bit**.
- A PWM signal consists of two main components that define its behavior:
 - a **duty cycle** and
 - a **frequency**.

-
- Those pins are normal digital pins.
 - The timer input and outputs connected to those pins are something extra.
 - The Timers will only use the pins if that is programmed in the registers.

-
- Because each of those timers has two compare registers, and so has the ability to control two pins with pwm.
 - 6 pins for 3 timers
 - The pwm is hardware, it doesn't use the interrupt.
 - you can create a "software pwm" using the interrupt routine

-
- ATmega328內部有3個計時計數器，
 - timer0 控制5、6的PWM訊號，以及delay()、millis()、micros()等指令。
 - timer1 控制9、10的PWM訊號，以及servo.h。
 - timer2 控制11、3的PWM訊號，以及tone()指令。

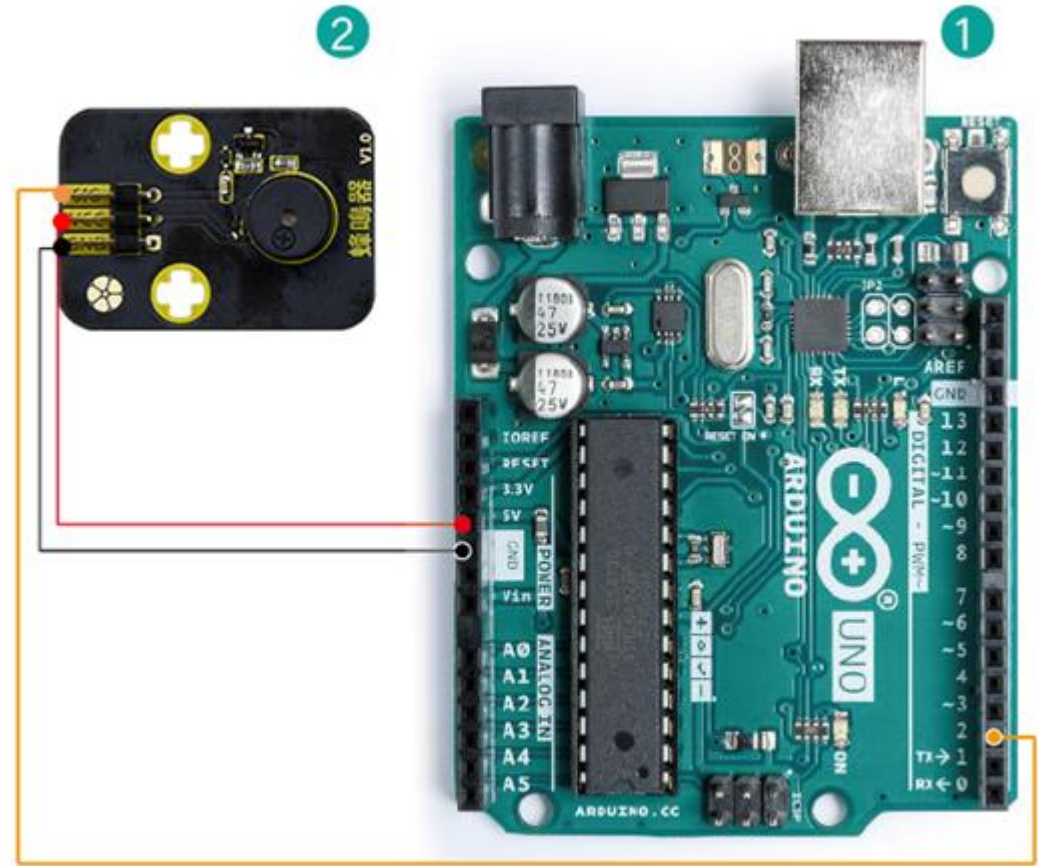
Arduino冷知識 2



圖/文 黃信惠的瘋狂教室

3Pin Buzzer

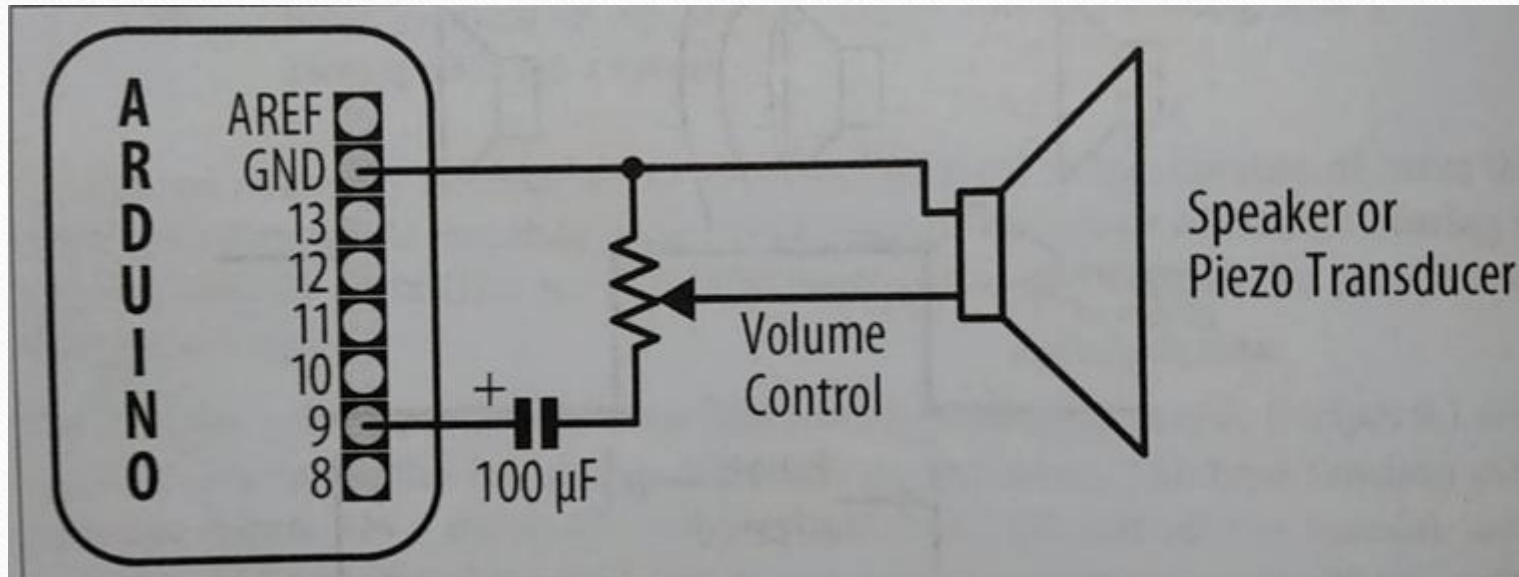
V (VCC) : 電源
G (GND) : 接地
S (STB) : PWM pin



-
- Arduino uses `TIMER0` for the timing.
 - You should not change that.
 - You can do whatever you want with `TIMER1` and `TIMER2`.
 - Some functions or libraries use those timers (`pwm`, `tone()`, and many more), so you have to be careful how they are used.

The Circuit for connecting a Speaker or Piezo to an Arduino pin

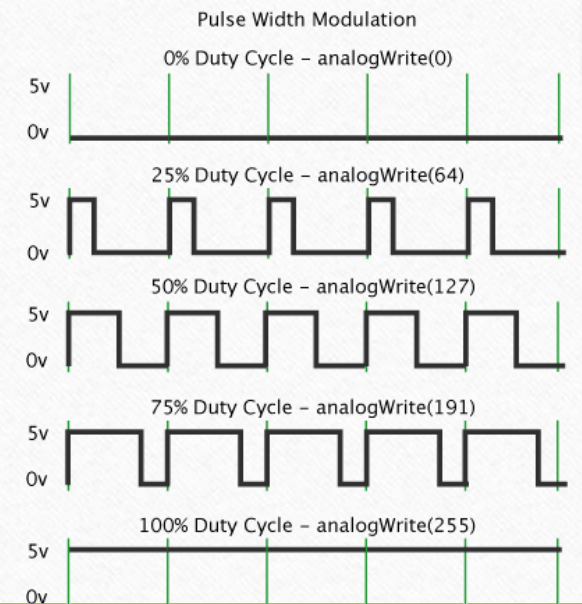
- Connecting to an audio transducer
 - The volume control is a variable resistor
-
- 200 to 500 ohms
 - A speaker will work regardless of which wire is attached to ground.
 - A Piezo buzzer is polarized, so connect the negative wire to the GND pin.



*timer1 控制9、10的PWM訊號，以及servo.h。
timer2 控制11、3的PWM訊號，以及tone()指令。*

Playing Tones with tone()

- Generates a square wave of the specified frequency (and 50% duty cycle) on a pin.
- A duration can be specified, otherwise the wave continues until a call to noTone().



tone()

- Use of the tone() function will interfere with PWM output on pins 3 and 11.
- It is not possible to generate tones lower than 31Hz.

tone(pin, frequency)

tone(pin, frequency, duration)

- pin:
 - the Arduino pin on which to generate the tone.
- frequency:
 - the frequency of the tone in hertz.
 - Allowed data types: unsigned int.
- duration:
 - the duration of the tone in milliseconds (optional).
 - Allowed data types: unsigned long.

```
1  /*
2   * Twinkle sketch
3   * Plays "Twinkle, Twinkle Little Star"
4   * Speaker is connected to digital pin 9
5   */
6
7  const int speakerPin = 9; // connect speaker to pin 9
8
9  char noteNames[] = {'C','D','E','F','G','a','b'};
10 unsigned int frequencies[] = {262,294,330,349,392,440,494};
11 const byte noteCount = sizeof(noteNames); // number of notes (7 here)
```



```
13 //notes, a space represents a rest
14 char score[] = "CCGGaaGFFEEDDC GGFFEEDGGFFEED CCGGaaGFFEEDDC "; //sheet music樂譜
15 const byte scoreLen = sizeof(score); // the number of notes in the score
16
17 void setup()
18 {
19 }
```

```
21 void loop()
22 {
23     for (int i = 0; i < scoreLen; i++)
24     {
25         int duration = 333; // each note lasts for a third of a second
26         playNote(score[i], duration); // play the note
27         delay(duration/10); // slight pause to separate the notes
28     }
29
30     delay(4000); // wait four seconds before repeating the song
31 }
```

```
33 void playNote(char note, int duration)
34 {
35     // play the tone corresponding to the note name
36     for (int i = 0; i < noteCount; i++) //搜尋要彈奏哪個音的代碼，不是好方法
37     {
38         // try and find a match for the noteName to get the index to the note
39         if (noteNames[i] == note) // find a matching note name in the array
40             tone(speakerPin, frequencies[i], duration); // play the note
41     }
42     // if there is no match then the note is a rest, so just do the delay
43     delay(duration);
44 }
```


-
- If you want to specify the length of each note, add the following variables
 - *beats*
 - an array showing the length of each note
 - 1 is an eighth note
 - 2 is a quarter note
 - *beat*
 - the number of beats per minute //每分鐘要彈奏幾個八分之一音符
 - *speed*
 - convert beats per minute into a duration in milliseconds. //一個八分之一音符要持續幾個ms

```
byte beats[scoreLen] = {1,1,1,1,1,1,2, 1,1,1,1,1,1,2,1, //每個音符要持續幾個八分之一音符  
                        1,1,1,1,1,1,2, 1,1,1,1,1,1,2,1,  
                        1,1,1,1,1,1,2, 1,1,1,1,1,1,2};  
  
byte beat = 180; // beats per minute for eighth notes //每分鐘要彈奏幾個八分之一音符  
unsigned int speed = 60000 / beat; // the time in ms for one beat // 一分鐘有60000ms
```

```
27 void loop()  
28 {  
29     for (int i = 0; i < scoreLen; i++)  
30     {  
31         int duration = beats[i] * speed; // use beats array to determine duration  
32         playNote(score[i], duration); // play the note  
33         delay(duration/10); // slight pause to separate the notes  
34     }  
35  
36     delay(4000); // wait four seconds before repeating the song  
37 }
```

Generating Audio Tones without Interfering with PWM

generate the tone in software instead of with a timer

Analog Output

- analogWrite
 - not truly analog
 - uses a technique called **Pulse Width Modulation (PWM)** to emulate an analog signal using digital pulses.
- PWM
 - varies the proportion of the pulses' on time and off time
 - Low-level output: produces pulses that are **ON** more short period of time
 - High-level output: produces pulses that are **ON** more than they are off.

Pulse Width Modulation

0% Duty Cycle - `analogWrite(0)`



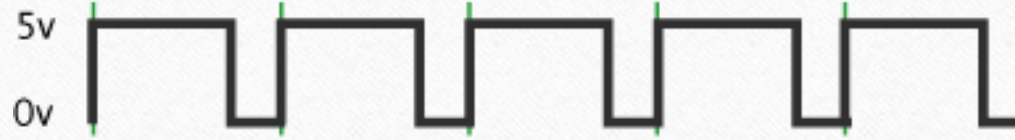
25% Duty Cycle - `analogWrite(64)`



50% Duty Cycle - `analogWrite(127)`



75% Duty Cycle - `analogWrite(191)`



100% Duty Cycle - `analogWrite(255)`

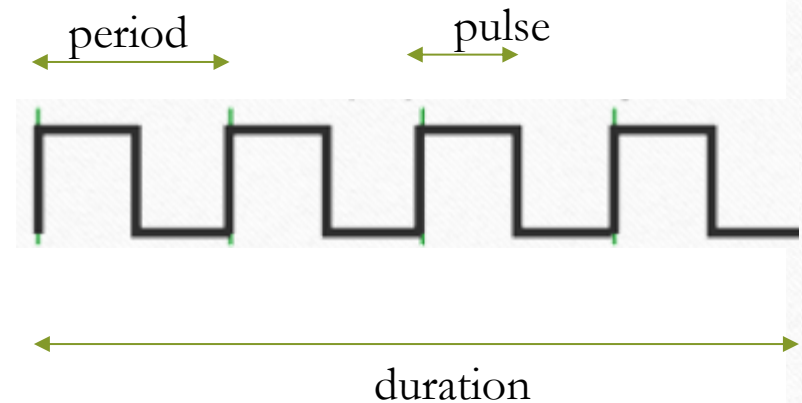


Generating Audio Tones Without Interfering with PWM

- When you need to use analogWrite on pin 3 or 11. (tied with timers)
 - timer2 控制11、3的PWM訊號，以及tone()指令
- Software solution (no timers are required)
 - Blocking
 - the code will not do anything else while the tone is played.
 - it will not return until the tone has finished.


```
1  /*
2   * Tone and fade sketch
3   * Plays tones while fading an LED
4   */
5  byte speakerPin = 9;
6  byte ledPin = 3;    // need to use analogWrite on pin 3
7                      //timer2 控制11、3的PWM訊號，以及tone()指令
8  void setup()
9  {
10     pinMode(speakerPin, OUTPUT);
11 }
```

```
13 void playTone(int period, int duration) // 自己產生方形波，驅動piezo device
14 {
15     // period is one cycle of tone //該音符的週期時間microseconds
16     // duration is how long the pulsing should last in milliseconds //該音符的彈奏時間ms
17     int pulse = period / 2; //方波半個週期的時間microseconds
18     for (long i = 0; i < duration * 1000L; i += period )
19     {
20         //1ms = 1000 microseconds
21         digitalWrite(speakerPin, HIGH);
22         delayMicroseconds(pulse);
23         digitalWrite(speakerPin, LOW);
24         delayMicroseconds(pulse);
25     }
```



```
27 void fadeLED(){
28     // These two static variables are assigned initial values
29     // only the first time the function is called.
30     static int brightness = 0;
31     static int changeval = 5;
32
33     analogWrite(ledPin, brightness);
34
35     // If we've exceeded the limits of analogWrite
36     brightness += changeval;
37     if (brightness >= 255 || brightness <= 0)
38         changeval *= -1; // Change direction
39
40     delay(2);
41 }
```



```
43  void loop()
44  {
45      // a note with period of 15289 is deep C (second lowest C note on piano)
46      for(int period=15289; period >= 477;  period=period / 2)  // play 6 octaves
47      {                                                           //八度
48          playTone(period, 200); // play tone for 200 milliseconds
49          fadeLED();
50      }
51  }
```

-
- The code stops and waits until a tone has completed before it can do any other processing.
 - It is possible to produce the sound in the background (without waiting for the sound to finish) by putting the sound generation code in an **interrupt handler**.